

Lecture 9: Recursion



INFO 1100: Introduction to Programming

what if a function
calls itself?

what if a function
calls itself?

+ *why might we want that?*

Recursion

Recursion

can be scary 😈

Recursion

can be scary 

What do I want you to learn?

Recursion

Recursion

What is it?

Recursion

Recursive function

A function that contains a recursive call
(a call to itself)

Recursion

Recursive function

A function that contains a recursive call
(a call to itself)

Recursive call

A call from a function to itself

We can repeat code without a loop!

Recursion vs. Iteration

Recursion vs. Iteration

Recursion

Using recursive functions

Iteration

Using loops

Recursion vs. Iteration

Recursion

Using recursive functions

Repeat body of function

Iteration

Using loops

Repeat body of loop

Recursion vs. Iteration

```
def blast_off(n):  
    for n in range(n, 0, -1):  
        print(n)  
    print("BLAST OFF!")
```

Recursion vs. Iteration

```
def blast_off(n):  
    for n in range(n, 0, -1):  
        print(n)  
    print("BLAST OFF!")
```

Recursion vs. Iteration

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Parts of a recursive function

Parts of a recursive function

```
def blast_off(n):
```

```
    if n == 0:  
        print("BLAST OFF!")
```

```
    else:
```

```
        print(n)
```

```
        blast_off(n-1)
```

Base case

Parts of a recursive function

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Base case

- Doesn't make a recursive call

Parts of a recursive function

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Base case

- Doesn't make a recursive call
- Computes an answer on its own

Parts of a recursive function

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Base case

- Doesn't make a recursive call
- Computes an answer on its own
- Stops the recursion

Parts of a recursive function

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Recursive case

Parts of a recursive function

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Recursive case

- Makes a recursive call

Parts of a recursive function

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Recursive case

- Makes a recursive call
- Needs help (from itself!) to compute the answer

Parts of a recursive function

```
def blast_off(n):  
    if n == 0:  
        print("BLAST OFF!")  
    else:  
        print(n)  
        blast_off(n-1)
```

Recursive case

- Makes a recursive call
- Needs help (from itself!) to compute the answer
- Solves a subproblem

Example: **Factorial**

Example: **Factorial**

$n!$

Example: **Factorial**

$n!$

Counts the # ways to order n distinct items

Example: **Factorial**

$$n! = n * (n - 1) * \cdots * 2 * 1$$

Example: **Factorial**

$$n! = \prod_{i=1}^n i$$

Example: **Factorial**

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n \geq 1 \end{cases}$$

Example: **Factorial**

*How do we write this using **iteration**?*

Example: **Factorial**

Iterative

```
def factorial(n):  
    product = 1  
    for i in range(1, n+1):  
        product = product * i  
    return product
```

Example: **Factorial**

*How do we write this using **recursion**?*

Example: **Factorial**

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n \geq 1 \end{cases}$$

Example: **Factorial**

base case

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n \geq 1 \end{cases}$$

Example: **Factorial**

base case

$$n! = \begin{cases} 1, & n = 0 \\ n * (n - 1)!, & n \geq 1 \end{cases}$$

recursive case

Example: **Factorial**

Recursion

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)
```

Example: **Factorial**

Recursion

```
def factorial(n):
```

```
    if n == 0:  
        return 1
```

base case

1, $n = 0$

```
    else:
```

```
        return n * factorial(n-1)
```

Example: **Factorial**

Recursion

```
def factorial(n):
```

```
    if n == 0:
```

recursive case

```
        return 1
```

$n * (n - 1)!, \quad n \geq 1$

```
    else:
```

```
        return n * factorial(n-1)
```

```
def mystery(n) :  
    if n == 0 :  
        return 0  
  
    else :  
        return n + mystery(n-1)  
  
print(mystery(4) )
```

What prints?

```
def mystery(n) :  
    if n == 0 :  
        return 0  
    else :  
        return n + mystery(n-1)  
  
print(mystery(4) )
```

How many times is mystery called?

```
def mystery(n) :  
    if n == 0 :  
        return 0  
    else :  
        return n + mystery(n-1)  
  
print(mystery(4) )
```

Recursion over nested data

Recursion over nested data

The University

 Name

 Objects and Powers

 Seals

 The Great Seal

 The Corporate Seal

The Board of Trustees

 Powers

 Membership and Tenure

The President

The University Faculty

 Membership

 Functions

 Dean of Faculty

The Degrees Awarded

Outline of the
Cornell University
by-laws

Recursion over nested data

The University
 Name
 Objects and Powers
 Seals
 The Great Seal
 The Corporate Seal
The Board of Trustees
 Powers
 Membership and Tenure
The President
The University Faculty
 Membership
 Functions
 Dean of Faculty
The Degrees Awarded

```
cornell_bylaws = [ 'The  
University', [ 'Name',  
              'Objects and Powers',  
              'Seals', [ 'The Great  
Seal', 'The Corporate  
Seal' ] ], 'The Board of  
Trustees', [ 'Powers',  
             'Membership and Tenure' ],  
            'The President', 'The  
University Faculty',  
            [ 'Membership',  
              'Functions', 'Dean of  
Faculty' ], 'The Degrees  
Awarded' ]
```

Recursion over nested data

Let's print the nested
data as an outline!

Recursion over nested data

Iteration?

Recursion over nested data

Iteration?

We don't know how deep
the nesting goes!

Recursion over nested data

```
def indent(level):  
    return ' ' * level  
  
def print_outline(outline, level):  
    for item in outline:  
        if isinstance(item, list):  
            print_outline(item, level + 1)  
        else:  
            print(indent(level) + item)
```

Error Zone

Error Zone

Infinite Recursion

Infinite Recursion

Infinite Recursion

what happens when we never
reach the base case

Infinite Recursion

```
def uh_oh(n):  
    if n == 0:  
        print("Blast off")  
    else:  
        print(n)  
        uh_oh(n)
```

Infinite Recursion

```
def uh_oh(n):  
    if n == 0:  
        print("Blast off")  
    else:  
        print(n)  
        uh_oh(n) ← no n - 1
```

Infinite Recursion

```
def uh_oh(n) :  
    print(n)  
    uh_oh(n-1)
```

Infinite Recursion

```
def uh_oh(n):  
    print(n)  
    uh_oh(n-1)
```

no base case!

“Infinite” Recursion

“Infinite” Recursion

Python prevents it!

“Infinite” Recursion

Python prevents it!

RecursionError

Example 1

```
def power(base, exp) :  
    if exp == 0 :  
        return 1  
    return base * power(base, exp-1)  
  
print(power(2, 5))
```

Example 2

```
def stars(n):  
    if n == 0:  
        return  
    print("*" * n)  
    stars(n-1)  
  
print(stars(4))
```

Example 3

```
def is_palindrome(s):  
    if len(s) <= 1:  
        return True  
    if s[0] != s[-1]:  
        return False  
    return is_palindrome(s[1:-1])  
  
print(is_palindrome("racecar"))  
print(is_palindrome("hello"))
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/09 @ 11:30am)

- Lab 9
- Readings
Think Python –
10.1-10.7
- GRQs